Kubernetes Volumes, PV/PVC, StorageClass

Kubernetes Volumes — Full Explanation

Why do we need volumes in K8s?

Containers are ephemeral by design. When a container in a pod crashes or restarts, its filesystem is reset — anything written to the container's local filesystem is lost. Additionally, when a pod has multiple containers, they can't naturally share files with each other since each container has its own isolated filesystem.

This creates two core problems:

1. **Data persistence** — databases, logs, uploaded files, etc. need to survive container restarts and even pod rescheduling.
2. **Data sharing** — sidecar containers (e.g., a log-shipper alongside a main app) need a common place to read/write files.

Volumes solve this by decoupling storage from the container lifecycle. A volume is a directory (possibly backed by some storage medium) that's accessible to containers in a pod, with a lifecycle that can be tied to the pod, or completely independent of it.

What are Volumes?

A **Volume** in Kubernetes is a directory, possibly with data in it, accessible to the containers in a pod. It's declared in the pod spec (`spec.volumes`) and mounted into containers via `volumeMounts`. Unlike Docker volumes, K8s volumes are tied to the **pod's lifecycle by default** (unless it's a persistent-type volume) — meaning data survives container crashes/restarts but is lost when the pod itself is deleted (for ephemeral volume types).

Categories of Volumes

1. `emptyDir`

- Created when a pod is assigned to a node, and exists as long as the pod runs on that node.
- Starts **empty** — hence the name.
- Shared among all containers in the pod — great for scratch space or sidecar communication.
- Data is **lost when the pod is deleted** (not just container restart — container restarts are fine, pod deletion is not).
- Can be backed by node disk or `memory` (tmpfs) for speed.
- **Use case:** temp cache, scratch computation space, shared work directory between app + sidecar.

```
      volumes:
      - name: cache-volume
        emptyDir: {}
```

2. `hostPath`

- Mounts a file/directory from the **host node's filesystem** directly into the pod.
- Data persists on that specific node even if the pod is deleted — but it's tied to that node, so if the pod reschedules elsewhere, the data isn't there anymore.
- **Security risk:** gives pod access to the host filesystem, potential for privilege escalation, container breakout, or interfering with node-level files. Also breaks portability — behavior differs depending on what's on that specific node.

- **Use case (narrow):** node-level agents like log collectors, monitoring agents (e.g., accessing `/var/log` or `/var/run/docker.sock`), not general application storage.
- **Generally NOT preferred** for regular app data because it's not portable, not multi-node aware, and is a security concern.

```
    volumes:
    - name: host-vol
      hostPath:
        path: /data
        type: Directory
```

3. Cloud / Network Volumes (via CSI — Container Storage Interface)

- These are volumes backed by actual persistent storage systems: AWS EBS, GCE PD, Azure Disk, NFS, Ceph, Portworx, etc.
- **CSI (Container Storage Interface)** is a standardized plugin API that lets storage vendors write drivers that "plug into" Kubernetes (and other orchestrators) without needing code baked into Kubernetes core (this replaced the old in-tree volume plugins model).
  - Each storage vendor ships a **CSI driver** (usually deployed as a DaemonSet + Deployment combo — node plugin + controller plugin).
  - Kubernetes talks to the CSI driver via a standard gRPC interface for operations like `CreateVolume`, `DeleteVolume`, `Mount`, `Unmount`, `Attach`, `Detach`.
  - This decouples Kubernetes releases from storage vendor releases — vendors can update their drivers independently.
- **This is the preferred approach for production**, because:
  - Data isn't tied to a single node — volumes can be attached/detached and moved with the pod.
  - Supports dynamic provisioning (see below).
  - Vendor-agnostic, pluggable, actively maintained ecosystem.

Quick comparison — Preferred vs Not

Volume Type	Persistence	Portability	Preferred?
`emptyDir`	Pod lifetime only	N/A (local, ephemeral)	OK for scratch/sidecar use, not for real data
`hostPath`	Node lifetime	Poor (node-bound)	**Avoid** for app data; only for node-agents
Cloud/CSI (network storage)	Independent of pod/node	Excellent	**Preferred** for real persistent workloads

PersistentVolume (PV) and PersistentVolumeClaim (PVC)

Kubernetes separates **storage provisioning** from **storage consumption** using two objects:

PersistentVolume (PV)

- A **cluster-level resource** representing a piece of actual storage (e.g., an EBS volume, NFS share) — provisioned either by an admin or dynamically.
- It exists independently of any pod's lifecycle.
- Has attributes: capacity, access modes, storage class, reclaim policy, and the actual backend (CSI driver info).

PersistentVolumeClaim (PVC)

- A **request for storage** made by a user/application — "I need 10Gi, ReadWriteOnce, SSD-backed."
- Kubernetes binds the PVC to a matching PV (or dynamically creates one).
- Pods reference the **PVC**, not the PV directly, in their volume spec — this abstracts away the underlying storage details from the application.

```
    apiVersion: v1
```

```yaml
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: standard
---
# In the pod:
volumes:
- name: data
  persistentVolumeClaim:
    claimName: my-pvc
```

Think of it like: **PV = the actual disk in a warehouse**, **PVC = a purchase order/reservation for a disk of certain specs**, and the **pod = the person who uses whatever disk got assigned to their order.**

Access Modes

Access modes define how a volume can be mounted, and depend on what the underlying storage backend supports:

Mode	Meaning
RWO (ReadWriteOnce)	Volume can be mounted read-write by a **single node** (multiple pods on that same node can still use it in newer K8s versions with `ReadWriteOncePod` being stricter — single pod only)
ROX (ReadOnlyMany)	Volume can be mounted read-only by **many nodes** simultaneously
RWX (ReadWriteMany)	Volume can be mounted read-write by **many nodes** simultaneously
RWOP (ReadWriteOncePod)	Volume can be mounted read-write by a **single pod** only (stricter than RWO, added for cases like single-writer databases where even co-located pods shouldn't share)

- Block storage like AWS EBS / GCE PD typically only supports **RWO** (can't be attached to multiple nodes at once).
- Network filesystems like NFS, CephFS, EFS, Azure Files typically support **RWX**.
- Choose access mode based on your app's concurrency needs: a single-instance database → RWO is fine; a shared file store across replicas (e.g., a shared upload directory across many web pods) → needs RWX.

Static vs Dynamic Provisioning

Static Provisioning

- A cluster **admin manually creates PV objects** ahead of time, describing real storage that already exists (e.g., a pre-created EBS volume, an NFS export).
- Users create PVCs, and Kubernetes matches/binds them to one of these pre-existing PVs based on size/access mode/storage class match.
- **Downside:** admin has to anticipate demand and pre-provision; doesn't scale well operationally.

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-static
spec:
  capacity:
```

```
      storage: 10Gi
  accessModes:
    - ReadWriteOnce
  csi:
    driver: ebs.csi.aws.com
    volumeHandle: vol-xxxxxxxx
```

Dynamic Provisioning

- No pre-created PVs needed. Instead, a **StorageClass** defines *how* to provision storage on demand (which CSI driver, disk type, parameters, reclaim policy).
- When a user creates a PVC referencing that StorageClass, Kubernetes (via the CSI provisioner controller) **automatically creates a new PV** (and the actual backing disk in the cloud) that matches the request, and binds it.
- This is the **standard modern approach** — it's how most production clusters work (EKS, GKE, AKS all ship default StorageClasses).

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 20Gi
```

Here, just creating the PVC triggers automatic creation of a matching PV and the underlying cloud disk — no admin intervention needed.

Summary: Static vs Dynamic

	Static	Dynamic
Who creates the PV	Admin, manually	Automatically, by the CSI provisioner
Scalability	Poor — manual effort per volume	Excellent — on-demand
Common today?	Rare, used for special pre-existing storage	**Default/standard practice**

Putting it all together

1. Admin defines a **StorageClass** (or relies on cluster default) pointing to a **CSI driver**.
2. Developer creates a **PVC** requesting size + access mode.
3. Kubernetes either binds it to an existing matching **PV** (static) or dynamically provisions a new one via the CSI driver (dynamic).
4. Pod references the **PVC** in its volume spec, and the PVC's underlying storage gets mounted into the container.
5. For non-persistent needs (scratch space, sidecar sharing), use `emptyDir`. Avoid `hostPath` except for node-level system agents.

StorageClass — Brief Explanation

A **StorageClass** is a Kubernetes object that acts as a **template/blueprint for provisioning storage dynamically**. It tells Kubernetes *how* to create a PersistentVolume on-demand when a PVC asks for one — which storage backend (CSI driver) to use, what type of disk, what settings, and what happens to the volume after it's released.

Think of it like a **"menu option" for storage** — instead of an admin manually creating disks ahead of time, the StorageClass defines the recipe, and Kubernetes cooks up a new volume automatically whenever someone orders from that menu (i.e., creates a PVC referencing it).

Key fields

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com     # which CSI driver creates the volume
parameters:                       # driver-specific options
  type: gp3
  iops: "3000"
reclaimPolicy: Delete             # what happens to the PV when PVC is deleted
volumeBindingMode: WaitForFirstConsumer   # when to actually provision the volume
allowVolumeExpansion: true        # can PVC be resized later?
```

Field	Meaning
provisioner	Which CSI driver (or in-tree plugin) handles volume creation — e.g., `ebs.csi.aws.com`, `disk.csi.azure.com`, `pd.csi.storage.gke.io`
parameters	Driver-specific settings — disk type (gp3/io2/ssd), filesystem type, replication, encryption, etc.
reclaimPolicy	`Delete` (destroy the backing disk when PVC is deleted) or `Retain` (keep the disk around for manual recovery)
volumeBindingMode	`Immediate` (provision as soon as PVC is created) or `WaitForFirstConsumer` (wait until a pod actually needs it, so it's provisioned in the correct zone/node)
allowVolumeExpansion	Whether PVCs using this class can later be resized larger

Why it matters

- Enables **dynamic provisioning** — no admin has to pre-create PVs.
- Lets you offer **multiple tiers of storage** in one cluster (e.g., `fast-ssd`, `standard-hdd`, `archive`), and users just pick the one they need via `storageClassName` in their PVC.
- Most managed clusters (EKS, GKE, AKS) ship with a **default StorageClass** already set up, so if a PVC doesn't specify one, it just uses the default.

Simple flow

```
StorageClass (recipe)
     -> PVC references it ("give me 10Gi of fast-ssd")
     -> CSI provisioner creates a new PV + real disk automatically
     -> PVC binds to that PV
     -> Pod uses the PVC
```